Futures ML Platform

Deployment & Operations Guide

AAI-540 Group 8 | University of San Diego

Tickers: CL=F (Crude Oil), GC=F (Gold)

Models: LSTM, Transformer, BiLSTM-Attention

AWS Region: us-east-1

Bucket: labs-usd-01

S3 Prefix: AAI_540_group_8

Table of Contents

1. Prerequisites

Before running any deploy scripts, make sure the following are in place:

- Python 3.10+ with a virtual environment (.venv) at the project root
- AWS CLI configured (or credentials.txt with keys)
- Required Python packages: boto3, sagemaker, pyyaml, numpy, pandas, scikit-learn, torch, joblib

Install dependencies (if fresh clone)

```
python -m venv .venv
source .venv/bin/activate
pip install boto3 sagemaker pyyaml numpy pandas scikit-learn torch joblib
```

Note: All deploy scripts must be run from the scripts/ directory because they resolve the config path relative to cwd. Always cd into scripts/ first.

2. Project Structure

Key files relevant to monitoring and CI/CD:

```
scripts/
  deploy_sagemaker_monitoring.py     # Launch monitoring job + CloudWatch alarms
  deploy_sagemaker_pipeline.py       # Create/run CI/CD pipeline
  deploy_sagemaker_training.py       # Standalone training job (reference)
  deploy_sagemaker_backtest.py       # Standalone backtest job (reference)
  deploy_s3_upload.py                # Upload datasets + models to S3
  sagemaker_monitoring_processing.py # Container script: health checks
  sagemaker_pipeline_preprocessing.py# Container script: feature engineering
  sagemaker_pipeline_evaluation.py   # Container script: model evaluation
  sagemaker_training.py              # Container script: model training
  compute_baseline_stats.py          # Generate baseline stats for drift

configs/
  config_training.yaml               # Hyperparams, tickers, S3 prefix

files/
  models/{ticker}/                   # Trained .pth models + scalers
  models/{ticker}/baseline_stats.json# Drift detection reference stats
  dataset/                           # Train/val/test CSVs
```

3. AWS Credentials Setup

Go to your AWS Learner Lab, click 'AWS Details', and copy the credentials block. Paste it into ~/.aws/credentials so it looks like this:

```
[default]
aws_access_key_id=ASIA...
aws_secret_access_key=...
aws_session_token=...
```

Note: Learner Lab tokens expire every ~4 hours. If you get an ExpiredToken error, refresh your credentials and try again.

4. Upload Data to S3

Before running monitoring or the CI/CD pipeline, ensure all datasets, trained models, and baseline stats are uploaded to your S3 bucket.

```
cd scripts
../.venv/bin/python deploy_s3_upload.py
```

This uploads everything under files/ (datasets, models, baseline_stats.json) to s3://{bucket}/{prefix}/. The bucket and prefix come from configs/config_training.yaml. If you need to override them:

```
../.venv/bin/python deploy_s3_upload.py --bucket labs-usd-01 --prefix AAI_540_group_8
```

5. Monitoring System

The monitoring system runs as a SageMaker Processing Job. It downloads recent production data and trained models from S3, then executes four health checks that publish custom metrics to CloudWatch.

5.1 What the monitoring checks

The monitoring script runs four independent health checks per ticker (CL=F and GC=F):

1. Data Drift Detection (Kolmogorov-Smirnov Test)

Compares the distribution of each of the 20 features in production data against the training baseline. Uses the two-sample KS test: if the p-value is below 0.05, that feature has drifted. Features are grouped into four buckets (price_trend, momentum, volume, volatility) and the drift ratio per bucket is reported as a CloudWatch metric.

2. Structural Break Detection (CUSUM)

Applies the Cumulative Sum (CUSUM) algorithm to the close price series. If the cumulative deviation from the mean exceeds a threshold, it signals a structural break (e.g. a regime change in the market). A structural break means the statistical properties of the price series have fundamentally changed, which may invalidate model assumptions.

3. Model Disagreement Score

All three models (LSTM, Transformer, BiLSTM-Attention) generate predictions on the same production data. The coefficient of variation (std / mean) across the three predictions is computed. High disagreement (>0.10) indicates the models are uncertain or seeing data outside their training distribution.

4. Prediction Anomaly Detection

Each model's mean prediction is compared against the training baseline (mean +/- 2 standard deviations). If a prediction falls outside this range, it's flagged as anomalous. This catches cases where a model produces unrealistic forecasts.

5.2 CloudWatch Alarms

Five CloudWatch alarms are created, each monitoring a different aspect of data drift. They trigger when the drift ratio exceeds 0.15 (15% of features in that group have drifted).

Alarm Name	Feature Group	Features Monitored
futures-drift-price-trend	Price Trend	MA, EMA, KAMA, WMA, MidPrice, TSF
futures-drift-momentum	Momentum	BOP, CMO, MFI, ROC, WILLR
futures-drift-volume	Volume	AD, OBV, volume
futures-drift-volatility	Volatility	NATR, ATR, TRANGE
futures-drift-global	All Features	All 20 features combined

All metrics are published to CloudWatch namespace 'FuturesMLOps/Monitoring' with dimensions for Ticker (CL=F or GC=F) and CheckType.

5.3 How to run monitoring

```
cd scripts

# Run monitoring job (uses config defaults for bucket/prefix):
```

```
../.venv/bin/python deploy_sagemaker_monitoring.py

# Override bucket/prefix if needed:
../.venv/bin/python deploy_sagemaker_monitoring.py \
    --bucket labs-usd-01 --prefix AAI_540_group_8
```

The script will: (1) upload the monitoring container script to S3, (2) launch a SageMaker Processing Job, (3) poll until completion, (4) create the 5 CloudWatch alarms, and (5) download the monitoring report to files/monitoring/sagemaker_results/.

Expected runtime: ~3-4 minutes. Estimated cost: ~$0.01.

5.4 Reading the monitoring report

After the job completes, a JSON report is saved at files/monitoring/sagemaker_results/monitoring_report.json. It contains per-ticker results for each health check:

- drift_ratios: fraction of features drifted per bucket (0.0 = no drift, 1.0 = all drifted)
- structural_break: true/false, whether a regime change was detected
- model_disagreement: coefficient of variation across models (lower is better)
- prediction_anomalies: which models produced out-of-range predictions

5.5 Cleanup

```
# Delete CloudWatch alarms when done:
../.venv/bin/python deploy_sagemaker_monitoring.py --cleanup
```

6. CI/CD Pipeline

The CI/CD pipeline automates the full model retraining workflow. It is implemented as a SageMaker Pipeline: a sequence of steps that SageMaker orchestrates, spinning up and tearing down instances for each step automatically.

6.1 Pipeline steps explained

Step 1: Preprocess

A Processing Job that: installs TA-Lib, loads raw OHLCV data from S3, computes the 20 technical indicator features (same as local training), and splits the data into train (60%), validation (20%), and test (20%) sets. Data is saved UNSCALED - the training step handles scaling.

Step 2: Train

A Training Job using the sagemaker_training.py entry point (same script as standalone training). Trains the specified model (default: LSTM) on the specified ticker (default: CL=F) for N epochs (default: 5 for validation, 200 for production). Outputs the trained model .pth file, scalers, and metadata as model.tar.gz.

Step 3: Evaluate

A Processing Job that loads the trained model and scalers from Step 2, scales the test data from Step 1, runs inference, and computes regression metrics: MSE, RMSE, MAE. Writes evaluation.json for the quality gate.

Step 4: CheckModelQuality (Condition)

A quality gate that reads the MSE from evaluation.json. If MSE <= threshold (default: 0.05 for validation, 0.005 for production), the model passes and proceeds to registration. Otherwise, the pipeline fails.

Step 5a: RegisterModel (if passed)

Registers the trained model in the SageMaker Model Registry under the appropriate package group (futures-cl-models for CL=F, futures-gc-models for GC=F) with status 'PendingManualApproval'. This creates a versioned record of the model for governance and deployment tracking.

Step 5b: FailModelQuality (if failed)

If MSE exceeds the threshold, the pipeline ends with an explicit failure message indicating the model did not meet quality standards.

6.2 Pipeline flow

```
[Preprocess] --> [Train] --> [Evaluate] --> [CheckModelQuality]
                                                    |           |
                                             MSE <= 0.05  MSE > 0.05
                                                    |           |
                                            [RegisterModel] [Fail]
```

6.3 How to run the pipeline

```
cd scripts

# Step 1: Register pipeline (no execution, just validates the definition):
../.venv/bin/python deploy_sagemaker_pipeline.py \
    --bucket labs-usd-01 --prefix AAI_540_group_8

# Step 2: Execute validation run (cheap: 5 epochs, 1 ticker, lenient MSE):
```

```
../.venv/bin/python deploy_sagemaker_pipeline.py \
    --bucket labs-usd-01 --prefix AAI_540_group_8 --execute

# Production run (full training, strict quality gate):
../.venv/bin/python deploy_sagemaker_pipeline.py \
    --bucket labs-usd-01 --prefix AAI_540_group_8 \
    --execute --epochs 200 --mse-threshold 0.005
```

6.4 CLI flags reference

Flag	Default	Description
--epochs	5	Training epochs (5=validation, 200=production)
--ticker	CL=F	Ticker to train (CL=F or GC=F)
--model	lstm	Architecture: lstm, transformer, bilstm_attention
--mse-threshold	0.05	Quality gate MSE ceiling (0.005 for prod)
--instance-type	ml.m5.large	SageMaker instance type
--execute	off	Actually run the pipeline (not just register)
--cleanup	off	Delete the pipeline definition
--bucket	auto	S3 bucket override
--prefix	auto	S3 prefix override

6.5 Cleanup

```
# Delete the pipeline definition from SageMaker:
../.venv/bin/python deploy_sagemaker_pipeline.py --cleanup
```

7. Cleanup Checklist

After validation, clean up AWS resources to avoid unnecessary charges. Run these from the scripts/ directory:

- CloudWatch alarms: ../.venv/bin/python deploy_sagemaker_monitoring.py --cleanup
- CI/CD pipeline: ../.venv/bin/python deploy_sagemaker_pipeline.py --cleanup
- S3 data (optional): aws s3 rm s3://labs-usd-01/AAI_540_group_8/pipeline-output/ --recursive

Note: Learner Lab resources are automatically cleaned up when the lab session ends. But it's good practice to clean up explicitly to keep costs predictable.

8. Cost Reference

Estimated costs for each operation (us-east-1, ml.m5.large @ $0.115/hr):

Operation	Duration	Est. Cost
Monitoring job	3-4 min	~$0.01
Pipeline validation (5 epochs)	15-20 min total	~$0.10-0.30
Pipeline production (200 epochs)	45-60 min total	~$0.50-1.00
Standalone training (all models)	30-60 min	~$0.30-0.70
S3 storage (datasets + models)	Ongoing	< $0.01/month

Note: Processing jobs (Preprocess, Evaluate, Monitoring) include ~2-3 min overhead for container startup and TA-Lib compilation. Training jobs include ~1 min overhead.

9. Troubleshooting

ExpiredToken error

Your AWS Learner Lab session token expired. Go to the Learner Lab console, copy fresh credentials, and paste into ~/.aws/credentials.

Config file not found

All deploy scripts resolve the config path as ../configs/config_training.yaml relative to the current working directory. Make sure you cd into scripts/ before running.

Pipeline step fails with AlgorithmError

This means the container script crashed. Check CloudWatch Logs under /aws/sagemaker/ProcessingJobs or /aws/sagemaker/TrainingJobs for the specific job name. The job name is printed in the deploy script output.

Model file not found in evaluation

The evaluation script extracts model.tar.gz automatically. If it fails, the training step may not have produced the expected output structure. Check that sagemaker_training.py saves models to /opt/ml/model/{ticker}/.

Wrong bucket or prefix

The default bucket/prefix come from configs/config_training.yaml. Use --bucket and --prefix flags to override. Double-check with: aws s3 ls s3://your-bucket/your-prefix/